

Job Search Command Center

Voice-first job search console · Project documentation

APPROACH

I treated the job search as one stateful loop — discover, apply, track, schedule, practise — and made voice the primary way to drive it, with the screen as a live view of that same state rather than a separate application. Every action the assistant takes is a real database write the interface reflects immediately, so speaking and clicking are interchangeable rather than parallel systems that drift apart.

SOLUTION

A FastAPI service over SQLite holds the domain: the résumé profile, fetched jobs, Gemini match scores, application stages and interviews. A Next.js client joins a LiveKit room, and a LiveKit Agents worker runs a Gemini Live speech-to-speech agent in that same room. Both processes share the database directly, so an action taken by voice appears on the dashboard without any intermediate synchronisation layer.

The agent is a single co-pilot with two modes rather than two personas. Mode is ordinary local state: entering a mock interview swaps the instructions, narrows the exposed tools to the interview's own two, and clears the conversation so the interviewer is genuinely isolated from earlier chat. This replaced an agent-handoff design that the realtime SDK silently discards whenever the user interrupts mid-sentence, which stranded users inside the interview with no way back.

Two decisions carry most of the reliability. Before each turn the current job feed is injected with its database ids, and every write tool verifies the id it was given and names the row it actually touched — previously the model invented ids and reported changes it had never made. Job lookup ranks fuzzily instead of filtering exactly, so a speech-to-text error like Senas still resolves to Sanas rather than failing silently.

The interface also drives the agent over LiveKit's data channel, giving buttons a path that never depends on the model choosing to act.

Stack Next.js 16 · FastAPI · SQLite · LiveKit Agents · Gemini Live (Vertex AI) · Adzuna · Google Calendar via MCP

213 automated tests · backend `pytest`, frontend `tsc / eslint` / next `build`